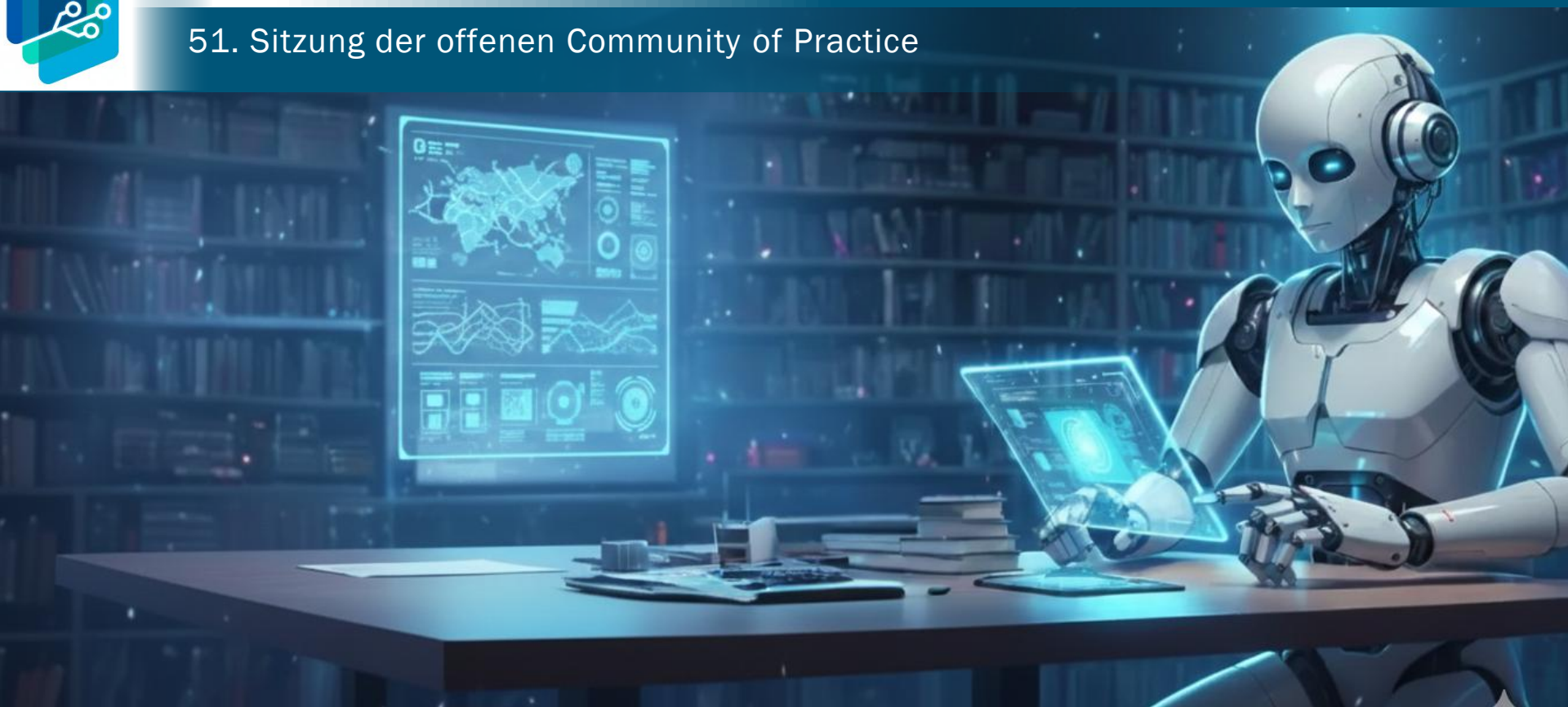




51. Sitzung der offenen Community of Practice



- **Update** (allgemein & persönlich)
- **Input:** „Logprobs & Grammar-based Sampling bei Textgenerierung & -analyse “
- **Diskussion**

Update (allgemein)



Arena Score German

selected models on lmarena.ai as of Oct 02, 2026

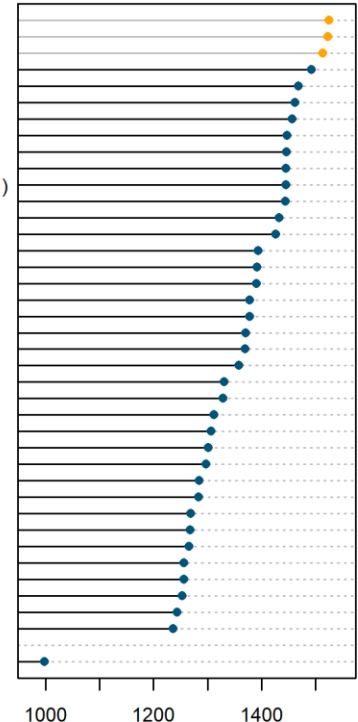
Hintergrund:

- Rechts abgebildet ist auszugsweise das aktuelle LMArena-Leaderboard für die Kategorie „German“ mit Blick auf die besten open-weights-Modelle
- je 1 pro Modellfamilie, zzgl. kleine Modelle (<30B) sowie die Top-3 proprietären Modelle

Bemerkenswert:

- Freies Modell **glm-5-3-max** bleibt an der Spitze, knapp vor **gemini-3-pro**
- Effizientestes Open-Weights Modell bleibt **Qwen3.8-27b** auch wenn gemma-4 nach wie vor nicht ausgewiesen ist.

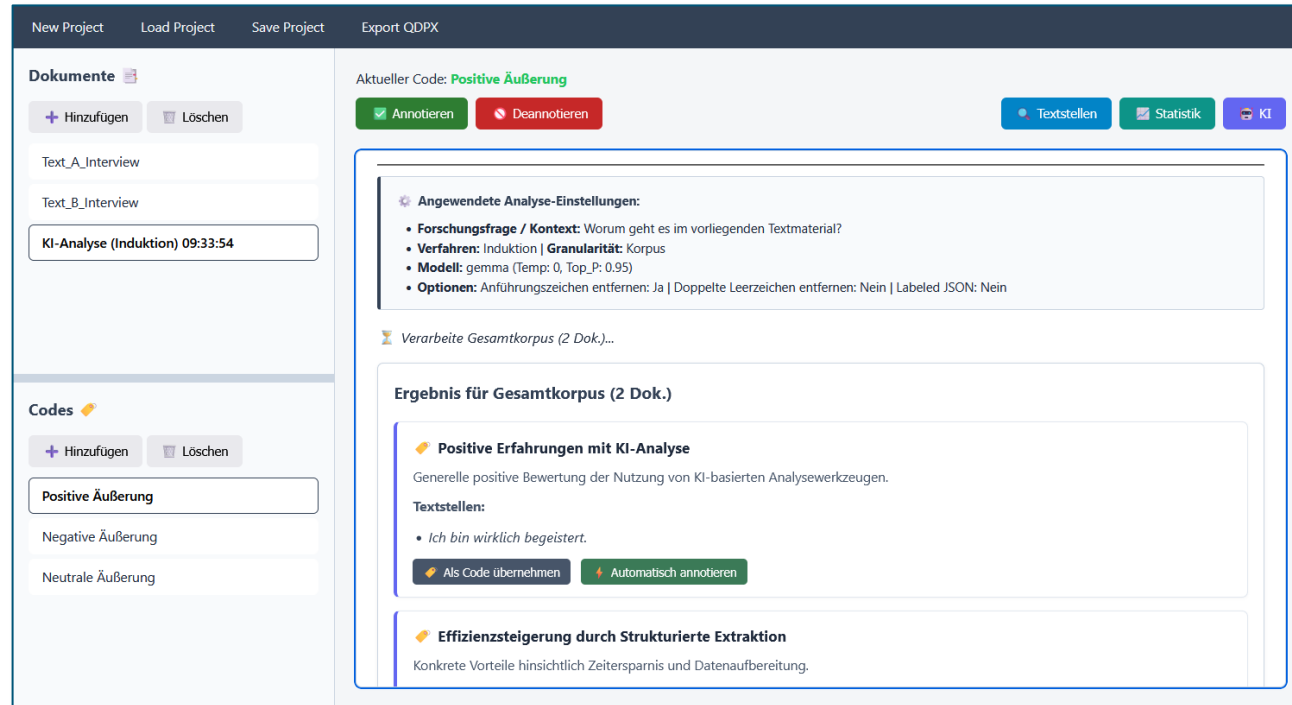
glm-5.3-max (Z.ai · MIT)
gemini-3-pro (Google · Proprietary)
muse-spark-1.3-max (Meta · Proprietary)
kimi-k3-max (Moonshot · Kimi K3 license)
deepseek-v4-pro (DeepSeek · MIT)
mimo-v2.5-pro (Xiaomi · MIT)
hy3 (Tencent · Apache 2.0)
inkling (Thinky · Apache 2.0)
qwen3.5-397b-a17b (Alibaba · Apache 2.0)
mistral-medium-3.5 (Mistral · Modified MIT)
nvidia-nemotron-3-ultra-550b-a55b-nvfp4 (Nvidia · OpenMDW-1.1)
qwen3.8-27b (Alibaba · Apache 2.0)
minimax-m3 (MiniMax · MiniMax Community License)
qwen3-235b-a22b-instruct-2507 (Alibaba · Apache 2.0)
step-3.5-flash (StepFun · Apache 2.0)
Inkling Small (Thinky · Apache 2.0)
qwen3.5-27b (Alibaba · Apache 2.0)
trinity-large-thinking (Apache 2.0)
longcat-flash-chat (Meituan · MIT)
gemma-3-12b-it (Google · Gemma)
gemma-3-27b-it (Google · Gemma)
command-a-03-2025 (Cohere · CC-BY-NC-4.0)
llama-4-maverick-17b-128e-instruct (Meta · Llama 4)
gpt-oss-120b (OpenAI · Apache 2.0)
llama-4-scout-17b-16e-instruct (Meta · Llama)
qwq-32b (Alibaba · Apache 2.0)
olmo-3.1-32b-instruct (AI2 · Apache 2.0)
mistral-small-3.1-24b-instruct-2503 (Mistral · Apache 2.0)
athene-70b-0725 (CC-BY-NC-4.0)
qwen-max-0919 (Alibaba · Qwen)
gemma-2-27b-it (Google · Gemma license)
gpt-oss-20b (OpenAI · Apache 2.0)
qwen2.5-72b-instruct (Alibaba · Qwen)
mistral-small-24b-instruct-2501 (Mistral · Apache 2.0)
phi-4 (Microsoft · MIT)
jamba-1.5-large (Jamba Open)
nemotron-4-340b-instruct (Nvidia · NVIDIA Open Model)
c4ai-aya-expense-32b (Cohere · CC-BY-NC-4.0)
...
qwen1.5-4b-chat (Alibaba · Qianwen LICENSE)



Update (persönlich) - MyQDA



- MyQDA unterstützt mit Version 1.16 nun auch überlappende Kodierungen
- MyQDA bietet manuelle und KI-basierte Kodierung (s.rechts), optimierte Modals (für Textstellen, Statistiken, Code-/Dokumentenbearbeitung usw.) und QDPX-Export



Quelle: <https://andreasfischer1985.github.io/code-snippets/html/MyQDA.html>

Structured Output & Logprobs

LMStudio chat-completions-endpoint:

- Über `response_format` lässt sich die Generierung auf syntaktisch mögliche Tokens begrenzen
- Über `logprobs` lassen sich log. Wahrscheinlichkeiten zu generierten Tokens angeben
- Über `top_logprobs` auch log. Wahrscheinlichkeiten konkurrierender Tokens

```
json_schema <- list(
  type = "json_schema",
  json_schema = list( name = "kategorie_array", schema = list(
    type = "array", items = list( type = "object",
      properties = list(
        code = list(type = "string"),
        description = list(type = "string"),
        textstellen = list(type = "array", items = list(type = "string")),
        required = c("code", "description", "textstellen"),
        additionalProperties = FALSE
      )
    )
  )
)
body <- list(
  model = model, messages = list(
    list(role = "system", content = "Extrahiere die Kategorien und Textstellen genau nach Vorgabe."),
    list(role = "user", content = prompt)),
  include=list("message.output_text.logprobs"),
  logprobs=TRUE, top_logprobs=6, max_tokens = 500, temperature = 0,
  stream = FALSE, response_format = json_schema
)
post <- httr::POST(
  url = url, body = body,
  httr::add_headers("accept" = "application/json", "Content-Type" = "application/json"),
  encode = "json"
)
```

Beispiel: Structured Output (gemma-4-12b-qat)

Prompt:

'Du bist ein wissenschaftlicher Assistent für qualitative Inhaltsanalysen nach Fischer (2025).

Kontext der Analyse: Zu bewerten sind Aussagen, die im Rahmen eines Interviews zur KI-basierten Inhaltsanalyse gemacht wurden.

Text:

"Befragte Person: Ich bin wirklich begeistert. Die strukturierte Extraktion mit MyQDA spart uns enorm viel Zeit bei der Aufbereitung des Korpus, und die Nachvollziehbarkeit der Kodierungen ist hervorragend."

Aufgabe: Ordne diese Äußerungen deduktiv je einer der folgenden Kategorien zu: (1) positive Äußerung, (2) neutrale Äußerung, (3) negative Äußerung.

Belege deine Zuordnung dabei mit passenden Textstellen.

Formuliere deine Antwort als JSON-Liste im folgenden Format:

```
[
  {
    "code": "Nummer der passende Kategorie",
    "description": "Code-Label",
    "textstellen": ["Exaktes wörtliches Zitat 1", "Exaktes wörtliches Zitat 2"]
  }
]
```



(Structured) Output:

```
[
  {
    "code": "1",
    "description": "positive Äußerung",
    "textstellen": [
      "Ich bin wirklich begeistert.",
      "Die strukturierte Extraktion mit MyQDA spart uns enorm viel Zeit bei der Aufbereitung des Korpus",
      "die Nachvollziehbarkeit der Kodierungen ist hervorragend"
    ]
  }
]
```

Implementation abhängig von Tool/Endpoint



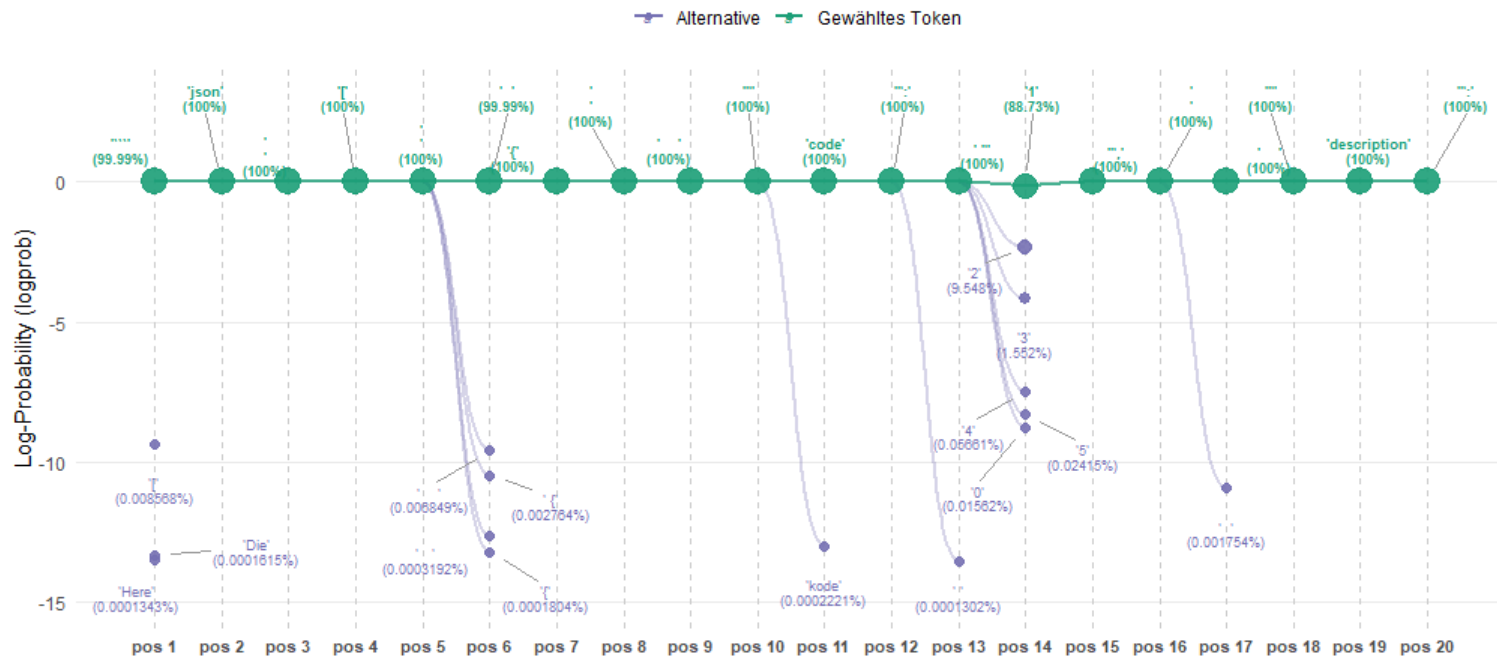
- Responses API erlaubt logprob-output
 - bei LMStudio aktuell aber noch kein grammar-based decoding
- Chat-Completions API erlaubt structured output (bzw. grammar-based decoding) & logprob-output

Logprobs (gemma-3-1b, unconstrained)



Verzweigungsbaum der Token-Wahrscheinlichkeiten

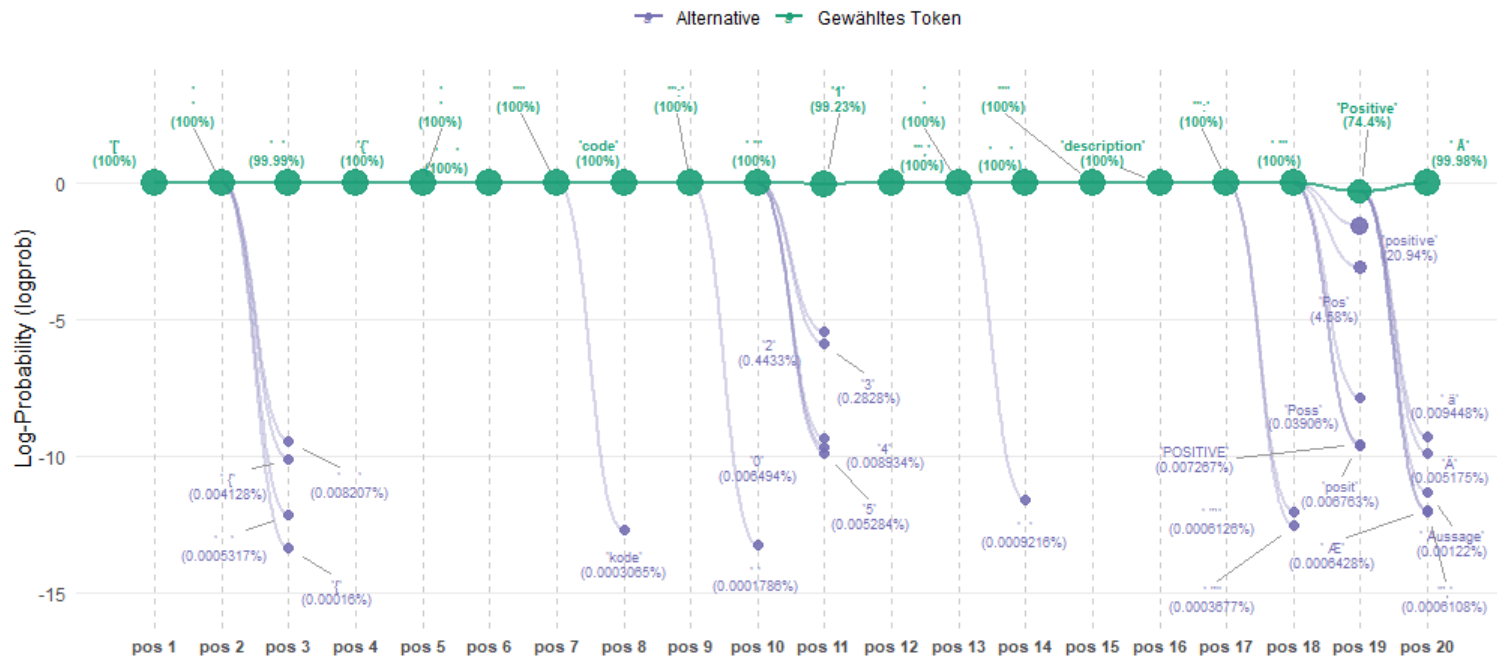
Vertikale Position = Log-Probability | Alternativen gefiltert ($p \geq 1e-04\%$)



Logprobs mit grammar-based-decoding

Verzweigungsbaum der Token-Wahrscheinlichkeiten

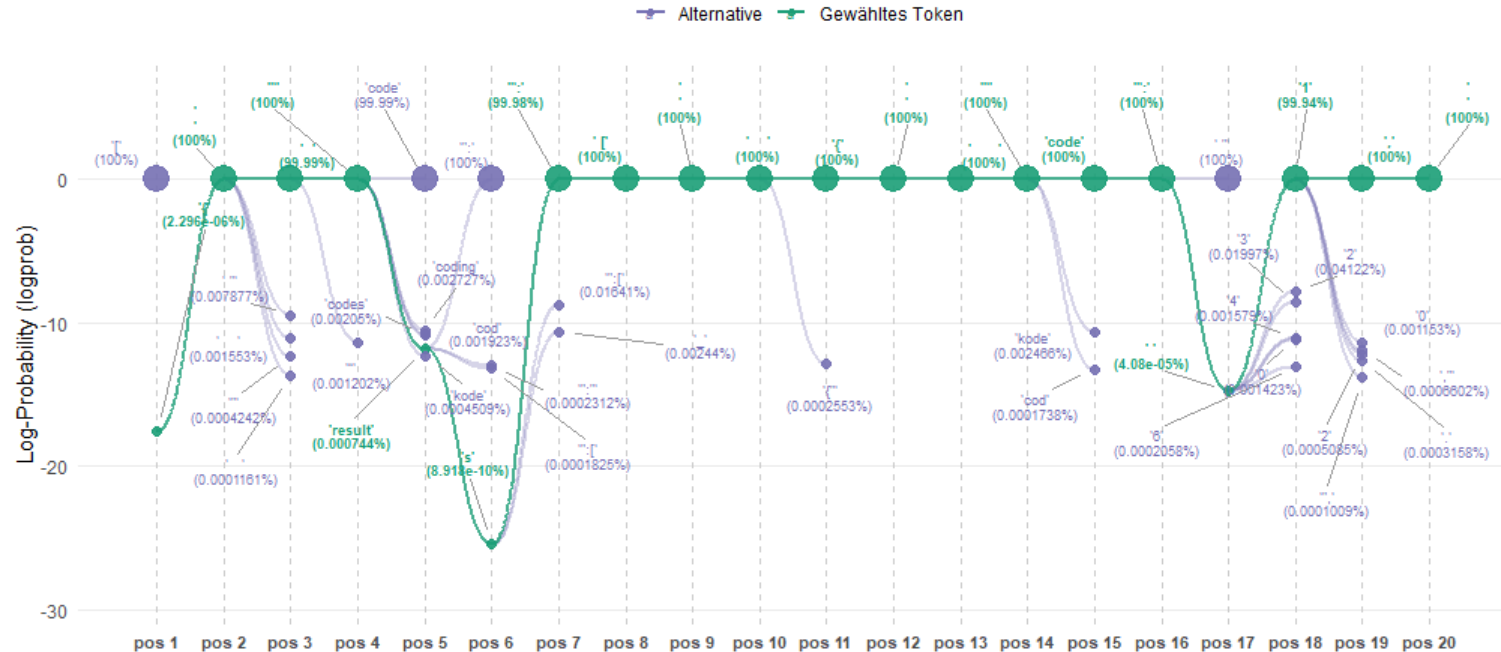
Vertikale Position = Log-Probability | Alternativen gefiltert ($p \geq 1e-04\%$)



Logprobs mit „fiesem“ grammar-based-decoding

Verzweigungsbaum der Token-Wahrscheinlichkeiten

Vertikale Position = Log-Probability | Alternativen gefiltert ($p \geq 1e-04\%$)

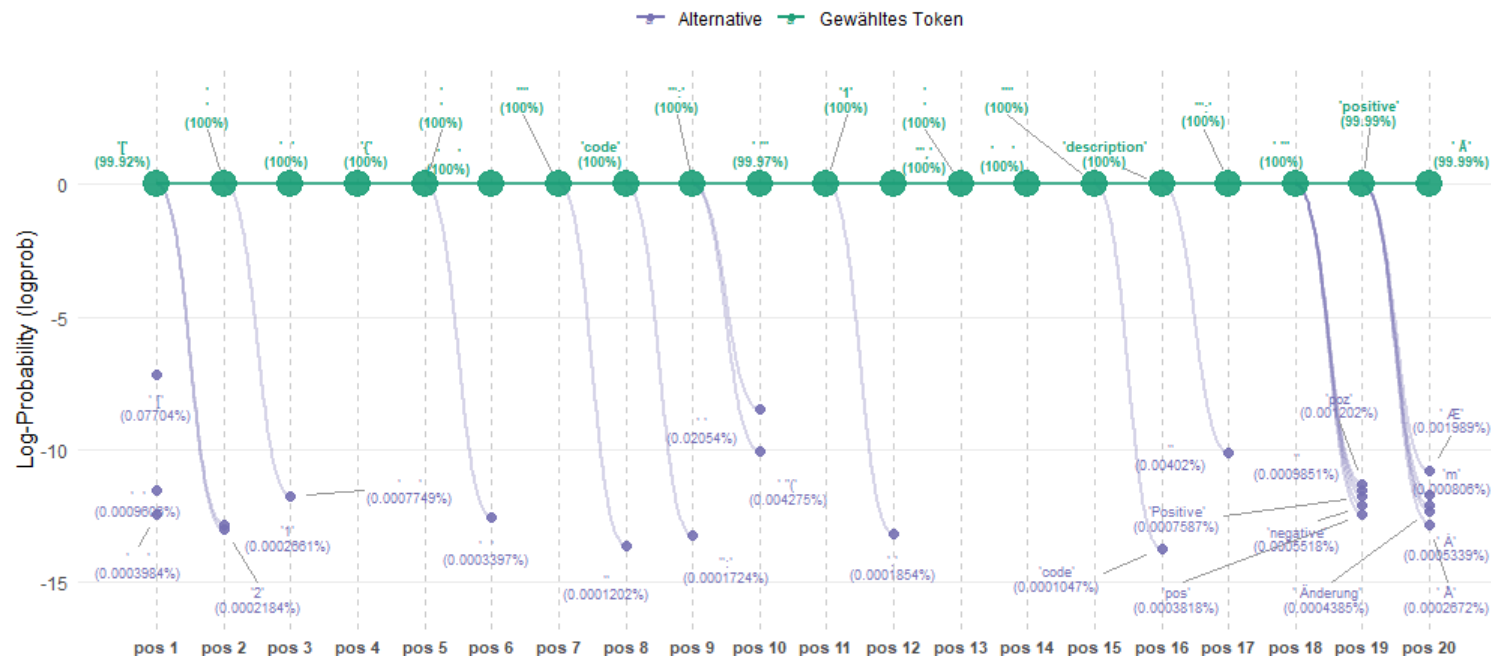


Logprobs mit grammar-based-decoding (gemma-4-12b-qat)



Verzweigungsbaum der Token-Wahrscheinlichkeiten

Vertikale Position = Log-Probability | Alternativen gefiltert ($p \geq 1e-04\%$)



- Für KI-basierte Inhaltsanalysen könnte man Logprobs heranziehen, um die geschätzte Wahrscheinlichkeit der gewählten Zuordnung im Vergleich zu diversen (validen oder invaliden) Alternativen zu beurteilen, am besten rekursiv ermittelt über separate paths aller wahrscheinlichen Folge-Tokens. Hier z.B. auf Basis von gemma-4-e4b-it:

```
> ergebnisse <- explore_token_paths(  
+   user_input = 'Du bist ein wissenschaftlicher Assistent für qualitative Inhaltsanalysen nach Fischer (2025).  
+   Kontext der Analyse: Zu bewerten sind Aussagen, die im Rahmen eines Interviews zur KI-basierten Inhaltsanalyse gemacht wurden.  
+   Text:\n"Befragte Person: Ich bin wirklich begeistert. Die strukturierte Extraktion mit MyQDA spart uns enorm viel Zeit bei der Aufbereitung des Korpus, uS  
+   [  
+     {  
+       "code": "Nummer der passende Kategorie",  
+       "description": "Code-Label",  
+       "textstellen": ["Exaktes wörtliches Zitat 1", "Exaktes wörtliches Zitat 2"]  
+     }  
+   ],  
+   initial_assistant = '[\n {\n   "code": ', # Vorgabe, ab der weitergeneriert wird  
+   threshold = -8, # Nur Tokens mit Logprob >= -7 berücksichtigen  
+   max_length = 2 # 3 Tokens tief weitersuchen  
+ )  
+ # Ergebnisse anzeigen  
+ print(ergebnisse)  
+  
+   text cumulative_logprob probability  
1 [\n {\n   "code": 1, -9.253686e-04 9.990751e-01  
2 [\n {\n   "code": 3, -7.543771e+00 5.293975e-04  
3 [\n {\n   "code": 2, -7.873997e+00 3.805103e-04  
4 [\n {\n   "code": 20 -1.330490e+01 1.666309e-06  
5 [\n {\n   "code": 30 -1.332101e+01 1.639680e-06  
6 [\n {\n   "code": 31 -1.475633e+01 3.903095e-07  
7 [\n {\n   "code": 2. -1.520545e+01 2.490907e-07  
8 [\n {\n   "code": 25 -1.534617e+01 2.163923e-07  
9 [\n {\n   "code": 21 -1.560911e+01 1.663602e-07
```

- Fragen?
- Anregungen?
- Erfahrungen?